GOEL INSTITUTE OF TECHNOLOGY AND MANAGEMENT LUCKNOW

DEPARTMENT OF COMPUTER SCIENCE

JAVA PROGRAMMING FOR BEGINNERS

SUMMER TRANNING REPORT (ONLINE TRANNING)

Submitted in Partial Fulfillment of the Requirements for the Award of

DIPLOMA IN COMPUTER SCIENCE

Submitted By

SACHIN CHAURASIYA

Enrollment No.: E24229135500051

Under the Guidance of

Krishna Kumar (CEO)

SimpleLearn SkillUP

Goel Institute of Technology and Management, Lucknow Board of Technical Education, Uttar Pradesh, Lucknow

Session – 2026-2027

CERTIFICATE



CERTIFICATE

This is to certify that Sachin Chaurasiya has successfully completed the online course “Java Programming for Beginners” offered by Simplilearn SkillUp. The course was completed on 4th September 2026, demonstrating his commitment towards learning and developing programming skills in Java. During the course, he gained knowledge of the fundamental concepts of Java programming and object-oriented programming. This certificate was issued by Simplilearn, with Krishna Kumar, CEO, Simplilearn, as the authorized signatory. The certificate is officially verified and bears the Certificate Code: 10690892. A copy of the original certificate is attached on the following page for verification and reference.

Name : Sachin Chaurasiya

Enrollment No : E24229135500051

DECLARATION

I hereby declare that this seminar report titled “Java Programming for Beginners” has been prepared by me as part of my academic and training work. The report presents the fundamental concepts studied during the online Java programming training and has been compiled for submission to Goel Institute of Technology and Management, Lucknow. I confirm that the material has been organized for educational purposes and that the programs and explanations included in this report are intended to demonstrate Java programming fundamentals such as variables, data types, control statements, arrays, strings, methods, recursion, and basic object-oriented programming. The online training was undertaken through SimpleLearn SkillUp under the guidance of Krishna Kumar (CEO). I accept responsibility for the correctness of the information and examples presented in this report.

Name : Sachin Chaurasiya

Enrollment No : E24229135500051

ABSTRACT

Java is a widely used programming language that provides a structured environment for learning core programming and object-oriented concepts. This seminar report presents the fundamentals of Java in a progressive sequence, beginning with the language overview and development environment and moving toward practical programming constructs. The report covers JDK, JRE and JVM, the structure of a Java program, keywords, identifiers, variables, constants, data types, type casting, console input and output, and operators. It then explains decision-making and repetition using conditional statements, switch, loops, break, and continue. A set of beginner number programs demonstrates how mathematical logic is converted into Java statements. The later sections introduce arrays, searching and sorting, two-dimensional matrices, strings, methods, method overloading, recursion, and the core principles of object-oriented programming. Small code examples are included to support understanding and practice. The objective is not only to describe syntax, but also to build a foundation for solving simple programming problems in a systematic manner.

ACKNOWLEDGEMENT

It is with great pleasure that I express my sincere thanks to everyone who supported me during the preparation of this seminar report titled “Java Programming for Beginners”. I am grateful to the Department of Computer Science, Goel Institute of Technology and Management, Lucknow, for providing the academic environment in which this work could be completed. I would especially like to thank Krishna Kumar (CEO), SimpleLearn SkillUp, for the guidance and instruction provided during the online Java programming training. The training helped me understand the basic syntax and programming ideas that form the foundation of this report. I also thank the faculty members of the Department of Computer Science for their encouragement, and my friends and classmates for their discussions and support while practicing Java programs. Finally, I am thankful to my family for their continued motivation and support throughout the preparation of this report.

Name : Sachin Chaurasiya

Enrollment No : E24229135500051

Table of contents

S. No.	Topic	Page No.
1	Introduction to Java – Objectives, Scope & Applications	8–9
2	Java Environment & Fundamentals – Features, JDK, JRE, JVM & Installation	9–10
3	Basic Java Programming – Program Structure, Variables, Data Types & Operators	10–22
4	Input/Output & Control Statements – Scanner, if-else, switch, Loops, break & continue	23–24
5	Pattern Programming	25
6	Methods, Arrays & Strings – Methods, Overloading, Arrays, Searching, Sorting, 2D Arrays & Strings	25–26
7	Object-Oriented Programming – Classes, Objects, Constructors, Encapsulation, Inheritance, Polymorphism, Abstraction & Interfaces	26–27
8	Packages & Access Modifiers	27–28
9	Advanced Core Java – Exception Handling, File Handling, Collections, Generics & Multithreading	28–29
10	Lambda Expressions & Stream API	30
11	Database Connectivity – JDBC, Architecture & Database Connection	31
12	CRUD Operations & Java Database Application Blueprint	32–33

Introduction to Java

- **Java** is a high-level, class-based, object-oriented programming language developed by **James Gosling** at Sun Microsystems in **1995** (now owned by Oracle). It was designed to have as few implementation dependencies as possible, allowing code to run seamlessly across different platforms.



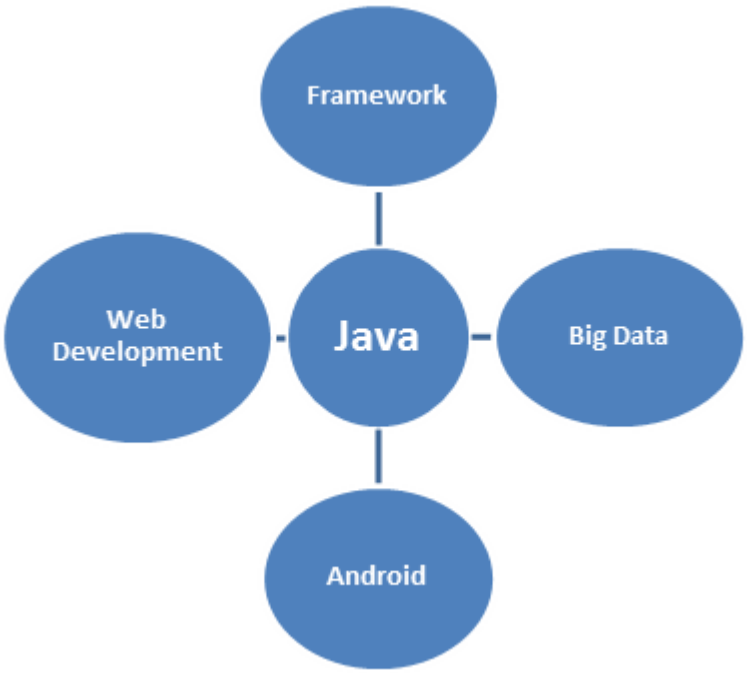
❖ Objectives of the Training

- The primary goals of mastering this Java syllabus are:
- **Core Mastery:** Understanding object-oriented logic, syntax rules, and data structures.
- **Problem Solving:** Developing structural logic for algorithms, searching, and sorting.
- **Enterprise Readiness:** Learning advanced concepts like database management, streams, and multithreading to build real-world backend applications.

❖ Scope of Java

- Java is highly versatile and dominates several massive tech domains:
- **Backend Web Development:** Powers massive enterprise applications (using Spring Boot).
- **Android Development:** The foundational language for native Android mobile applications.

- **Big Data:** Frameworks like Apache Hadoop and Apache Spark are built using Java.
- **Enterprise Applications:** Widely used in banking, retail, and financial software systems.



❖ Importance of Java in Software Development

- Java remains a top industry choice due to its scale and reliability:
- **Platform Independence:** The "Write Once, Run Anywhere" (WORA) philosophy saves development time.
- **Massive Ecosystem:** Millions of libraries, open-source frameworks, and tools exist.
- **Huge Community Support:** Decades of documentation mean almost any error has an online solution.
- **Backward Compatibility:** Older Java code runs smoothly on newer Java versions, making it highly sustainable for businesses.

Java Environment & Fundamentals

❖ Java Features

- **Simple:** Easy to learn; removes complex, error-prone C++ concepts like explicit pointers.
- **Platform Independent:** Compiles into architecture-neutral bytecode (.class files).
- **Object-Oriented:** Simulates real-world entities using objects, improving reusability.
- **Robust:** Focuses on compile-time and runtime error checking, automated memory management (Garbage Collection), and exception handling.
- **Secure:** Features like the bytecode verifier and sandbox isolation keep the host system safe.
- **Multithreaded:** Allows a program to perform multiple tasks concurrently for high performance.

❖ JDK, JRE, and JVM

- **JVM (Java Virtual Machine):** The engine that executes Java bytecode line-by-line. It is platform-dependent.
- **JRE (Java Runtime Environment):** The software layer containing the JVM and core libraries required to **run** Java applications.
- **JDK (Java Development Kit):** The full toolkit containing the JRE plus development tools like the compiler (javac) and debugger needed to **write** Java applications.
- **Formula:** JDK = JRE + Development Tools | JRE = JVM + Core Libraries

❖ Java Installation and Environment Setup

- **Download:** Get the latest JDK installer from the official Oracle website.
- **Install:** Run the setup file on your machine.

❖ Environment Variables (Windows):

- Create a new System Variable: Name = JAVA_HOME, Value = Path to your JDK installation (e.g., C:\Program Files\Java\jdk-xx).
- Edit the existing Path variable and add: %JAVA_HOME%\bin.

➢ **Test:** Run java -version and javac -version in your terminal to verify.

❖ Basic Structure of a Java Program

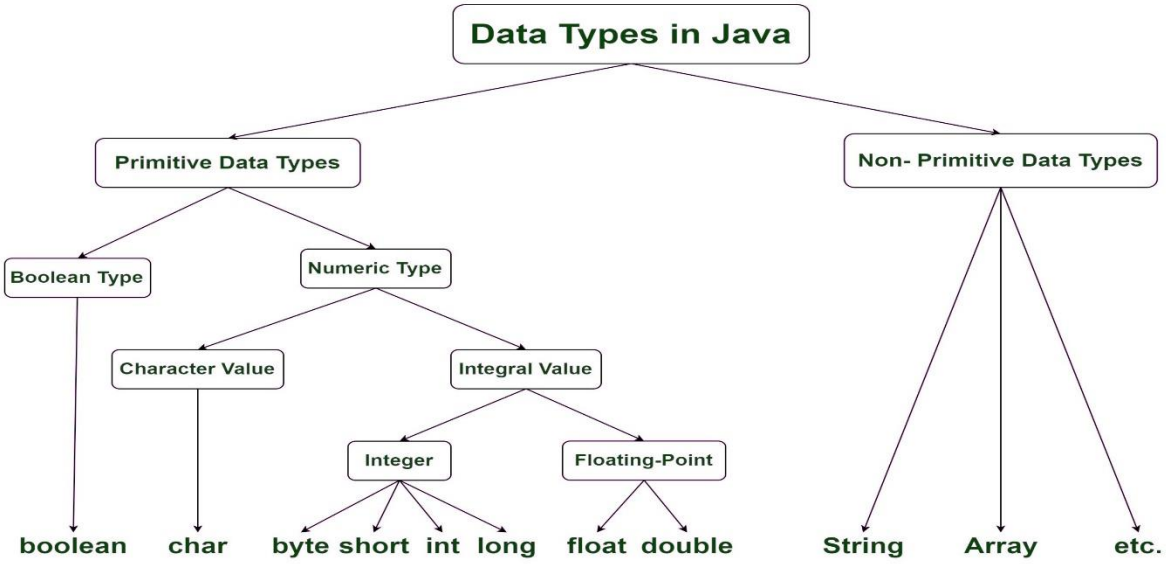
```
// Every file must have a class matching the filename exactly (e.g., Main.java)
public class Main {
// The mandatory entry point for any Java program
public static void main(String[] args) {
// A simple statement to print output
System.out.println("Hello, Java!");
}}
```

❖ Variables and Data Types

➢ **Variables:** Named memory locations used to hold data during execution.

➢ **Data Types:** Define the nature and size of data a variable can store:

- **Primitive Types:** Built-in types like byte, short, int, long (integers); float, double (decimals); char (characters); boolean (true/false).
- **Non-Primitive Types:** Reference types created by programmers, such as Strings, Arrays, Classes, and Interfaces.



❖ Operators

- Symbols used to execute operations on data values:
- **Arithmetic:** +, -, *, /, %
- **Relational:** ==, !=, >, <, >=, <=
- **Logical:** && (AND), || (OR), ! (NOT)
- **Assignment:** =, +=, -=, *=, /=
- **Unary:** ++ (Increment by 1), -- (Decrement by 1)

OPERATORS IN JAVA

❖ Introduction to Operators

- **Definition:** Operators in Java are special symbols used to perform operations on variables, constants, and values. They help programmers perform calculations, compare values, assign data, and make logical decisions.
- **Operands:** The values or variables on which an operator performs an operation are called operands. For example, in a + b, the + symbol is an operator, while a and b are operands.

- **Purpose of Operators:** Operators make Java programs easier to write and help perform mathematical calculations, comparisons, logical evaluations, and value modifications.
- **Classification:** Java provides different types of operators according to their functions. The important types covered in this section are:
- Arithmetic Operators
- Relational Operators
- Logical Operators
- Assignment Operators
- Unary Operators
- **Importance:** Operators are used in almost every Java program, including calculations, conditional statements, loops, data processing, and decision-making

❖ Arithmetic Operators

- Arithmetic operators are used to perform basic mathematical calculations on numeric values and variables. They are commonly used in programs involving marks, percentages, salaries, financial calculations, and other numerical operations.

Types of Arithmetic Operators

❖ Addition Operator (+)

- The addition operator is used to add two numeric values.
- It can operate on integer and floating-point values.
- It can also be used with strings to join text.
- Example: int sum = 10 + 5;
- Result: 15

❖ Subtraction Operator (-)

- The subtraction operator is used to calculate the difference between two numeric values.
- It subtracts the second operand from the first operand.
- It can produce a positive, negative, or zero result.
- Example: int result = 10 - 4;
- Result: 6

❖ Multiplication Operator (*)

- The multiplication operator multiplies two numeric values.
- It is useful for calculating totals, areas, and product quantities.
- Example: int product = 6 * 4;
- Result: 24

❖ Division Operator (/)

- The division operator divides one numeric value by another.
- When both operands are integers, Java performs integer division and removes the fractional part.
- When at least one operand is a floating-point value, the result can contain a decimal part.
- Division by zero with an integer operand causes an ArithmeticException at runtime.
- Example: int result = 10 / 3;
- Result: 3
- Example: double result = 10.0 / 3;
- Result: Approximately 3.3333

❖ Modulus Operator (%)

- The modulus operator returns the remainder after division.
- It is useful for checking whether a number is even or odd and for determining divisibility.
- Example: int remainder = 10 % 3;
- Result: 1
- Example: 10 % 2 returns 0, indicating that 10 is divisible by 2.
- In Java, the result follows the sign of the dividend, meaning the value on the left side of %.

❖ Applications of Arithmetic Operators

- Performing mathematical calculations in programs.
- Calculating total marks and percentage.
- Finding the area and perimeter of geometric shapes.
- Calculating salary, discounts, and expenses.
- Determining remainders and checking divisibility.
- Processing numerical data in applications.
- ---

❖ Relational Operators

- Relational operators are used to compare two values or expressions. They determine the relationship between operands and return a Boolean result: true or false.
- These operators are especially useful in decision-making statements such as if, if-else, and loops such as while and for.

Operators Types of Relational

❖ Equal to Operator (==)

- It checks whether two values are equal.
- It returns true if both operands are equal; otherwise, it returns false.
- Example: 10 == 10
- Result: true

❖ Not Equal to Operator (!=)

- It checks whether two values are different.
- It returns true if the operands are unequal.
- Example: 10 != 5
- Result: true

❖ Greater Than Operator (>)

- It checks whether the left operand is greater than the right operand.
- It returns true when the left value is greater.
- Example: 15 > 10
- Result: true

❖ Less Than Operator (<)

- It checks whether the left operand is smaller than the right operand.
- It returns true when the left value is smaller.
- Example: 5 < 10
- Result: true

❖ Greater Than or Equal to Operator (>=)

- It checks whether the left operand is greater than or equal to the right operand.
- It returns true when either condition is satisfied.
- Example: 10 >= 10
- Result: true

❖ Less Than or Equal to Operator (<=)

- It checks whether the left operand is smaller than or equal to the right operand.
- It returns true when either condition is satisfied.
- Example: 5 <= 10
- Result: true

❖ Applications of Relational Operators

- Comparing students' marks with passing marks.
- Checking whether a person meets an age requirement.
- Determining the largest or smallest number.
- Comparing prices, salaries, and quantities.
- Controlling program execution through conditional statements.
- Checking conditions in loops and other decision-making structures.
- **Important Note:** The equality operator == is different from the assignment operator =. The == operator compares values, whereas = assigns a value to a variable.

❖ Logical Operators

- Logical operators are used to combine or reverse Boolean conditions. They help a program make decisions based on one or more conditions.

- The main logical operators in Java are AND (&&), OR (||), and NOT (!).

Types of Logical Operators

❖ Logical AND Operator (&&)

- The AND operator returns true only when both conditions are true.
- If either condition is false, the complete expression returns false.
- It uses short-circuit evaluation, which means the second condition is not evaluated when the first condition is false.
- Example: (10 > 5) && (8 > 3)
- Result: true
- Example: (10 > 5) && (8 < 3)
- Result: false

❖ Logical OR Operator (||)

- The OR operator returns true if at least one condition is true.
- It returns false only when both conditions are false.
- It also uses short-circuit evaluation. If the first condition is true, the second condition is not evaluated.
- Example: (10 < 5) || (8 > 3)
- Result: true
- Example: (10 < 5) || (8 < 3)
- Result: false

❖ Logical NOT Operator (!)

- The NOT operator reverses the Boolean result of a condition.

- It changes true to false and false to true.
- It is a unary operator because it operates on a single Boolean expression.
- Example: !(10 > 5)
- Result: false
- Example: !(10 < 5)
- Result: true

❖ Applications of Logical Operators

- Checking multiple conditions in a single statement.
- Validating user input and login credentials.
- Applying eligibility criteria in applications.
- Checking whether a value lies within a particular range.
- Controlling loops based on multiple conditions.
- Implementing decision-making logic in Java programs.
- **Example:** A student may be eligible for a scholarship when their marks are at least 80 and their attendance is at least 75. The AND operator can combine these two conditions.

❖ Assignment Operators

- Assignment operators are used to assign values to variables. The simple assignment operator is =, while compound assignment operators combine an arithmetic operation with assignment.

Types of Assignment Operators

❖ Simple Assignment Operator (=)

- It assigns the value on the right side to the variable on the left side.
- The variable must be declared with an appropriate data type before assignment.
- Example: int a = 10;
- Result: The variable a stores the value 10.

❖ Addition Assignment Operator (+=)

- It adds the right-side value to the existing value of the variable and assigns the result back to that variable.
- Example: int a = 10; a += 5;
- Result: a becomes 15.
- Equivalent expression: a = a + 5;

❖ Subtraction Assignment Operator (-=)

- It subtracts the right-side value from the current value of the variable.
- Example: int a = 10; a -= 3;
- Result: a becomes 7.
- Equivalent expression: a = a - 3;

❖ Multiplication Assignment Operator (*=)

- It multiplies the current value of the variable by the right-side value.
- Example: int a = 6; a *= 4;
- Result: a becomes 24.

- Equivalent expression: a = a * 4;

❖ Division Assignment Operator (/=)

- It divides the current value of the variable by the right-side value and stores the result.
- Example: int a = 20; a /= 4;
- Result: a becomes 5.
- Equivalent expression: a = a / 4;
- When the variable and divisor are integers, integer division rules apply.

❖ Advantages of Assignment Operators

- They reduce repetitive code.
- They make mathematical updates easier to read.
- They are useful for updating counters and totals.
- They help modify existing variable values during program execution.
- They are commonly used in loops, calculations, and data processing.
- **Important Note:** The compound assignment operators perform an operation and assign its result to the left-side variable. They can have specific type-conversion behavior in Java, so they are not always exactly interchangeable with manually expanded expressions for every data type.

❖ Unary Operators

- Unary operators perform operations on a single operand. They are used to increase or decrease values, reverse Boolean expressions, and modify numeric values.
- The increment (++) and decrement (--) operators are commonly used in Java for controlling counters and loops.

Types of Unary Operators

❖ Increment Operator (++)

- The increment operator increases the value of a numeric variable by one.
- It is useful when counting repetitions or moving to the next index in an array.
- Example: int a = 5; a++;
- Result: a becomes 6.
- The operator has two forms: pre-increment and post-increment.

❖ Pre-Increment (++a)

- The value is incremented before it is used in the surrounding expression.
- Example: int a = 5; int b = ++a;
- Result: a = 6 and b = 6.

❖ Post-Increment (a++)

- The original value is used in the surrounding expression before the variable is incremented.
- Example: int a = 5; int b = a++;
- Result: b = 5 and a = 6.

❖ Decrement Operator (--)

- The decrement operator decreases the value of a numeric variable by one.
- It is useful for countdowns, reverse iteration, and reducing counters.
- Example: int a = 5; a--;
- Result: a becomes 4.
- It also has pre-decrement and post-decrement forms.

❖ Pre-Decrement (--a)

- The value is decreased before it is used in the surrounding expression.
- Example: int a = 5; int b = --a;
- Result: a = 4 and b = 4.

❖ Post-Decrement (a--)

- The original value is used in the surrounding expression before the variable is decreased.
- Example: int a = 5; int b = a--;
- Result: b = 5 and a = 4.

❖ Applications of Unary Operators

- Increasing or decreasing loop counters.
- Traversing arrays using index values.
- Implementing countdowns and repetition-based tasks.
- Updating numeric variables efficiently.
- Supporting calculations that require a change of one unit.
- **Important Note:** The increment and decrement operators are applied to variables that can be modified. They cannot be applied directly to constant values such as 5++.

❖ Input and Output

- **Output:** Handled standardly by System.out.print() (same line) or System.out.println() (appends a newline).
- **Input:** Managed using the Scanner class from the java.util package.

```
import java.util.Scanner;
```

```
Scanner scanner = new Scanner(System.in);
System.out.print("Enter a number: ");
int num = scanner.nextInt(); // Reads integer input
scanner.close();
```

Control Statements

❖ Conditional Statements

- Used to execute code blocks based on specific conditions:
- **if / if-else:** Runs code only if a condition evaluates to true, or defaults to an else block.
- **if-else-if ladder:** Evaluates multiple conditions sequentially until one matches.

❖ Switch Statement

- Acts as an alternative to long if-else ladders. It tests a variable against multiple fixed values (cases).

```
int standard = 2;

switch(standard) {
case 1: System.out.println("First Class"); break;
case 2: System.out.println("Second Class"); break; // Executes
default: System.out.println("Unknown Class");
}
```

❖ Looping Statements

- Used to run a block of code multiple times based on a condition:
- **for loop:** Best used when the number of iterations is known in advance.

- **while loop:** Evaluates a condition *before* executing the block; best when iterations are unknown.
- **do-while loop:** Evaluates the condition *after* execution, guaranteeing the code runs **at least once**.

❖ Break and Continue Statements

- **break:** Instantly terminates the loop or switch block entirely, jumping past it.
- **continue:** Skips the remaining code in the current loop iteration and jumps straight to the next evaluation step.

❖ Pattern Programming

- Utilizes nested loops (a loop inside another loop) to construct shapes, matrices, or designs out of characters or numbers.

```
// Prints a simple 3x3 block of asterisks

for (int i = 1; i <= 3; i++) {       // Rows
for (int j = 1; j <= 3; j++) {   // Columns
System.out.print("* ");
}
System.out.println();
}
```

Methods, Arrays & Strings

❖ Methods in Java

- A method is a reusable block of code designed to execute a specific task. It only runs when it is called.

```
// AccessModifier ReturnType MethodName(Parameters)
public int addNumbers(int a, int b) {
```

return a + b;

}

❖ Method Overloading

- A feature that allows a class to have more than one method with the **same name**, provided their **parameter lists are different** (either by data type or number of arguments).
- int printData(int a) { return a; }
- double printData(double a) { return a; } *// Overloaded*

❖ Arrays

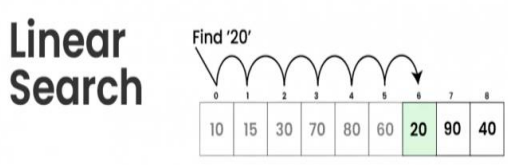
- An array is an indexed collection of variables of the **same data type** stored in contiguous memory locations. Arrays are fixed in size once declared.

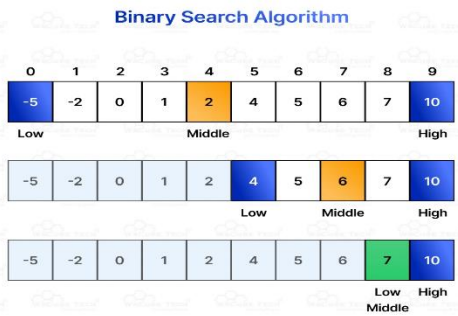
  int[] numbers = {10, 20, 30, 40};

  System.out.println(numbers[0]); *// Accesses the first element (10)*

❖ Searching and Sorting

- **Linear Search:** Checks elements one by one from start to finish.
- **Binary Search:** Faster search method that repeatedly divides a **sorted** array in half.





- **Sorting (e.g.), Bubble, Selection, Insertion):** Algorithms used to arrange elements in ascending or descending order.

❖ Two-Dimensional Arrays

- An array of arrays, represented structurally as a matrix of **rows and columns**.

```
int[][] matrix = {
{1, 2},
{3, 4}
};

System.out.println(matrix[0][1]); // Prints row 0, column 1 (Value: 2)
```

❖ Strings

- In Java, a String is an object that represents a sequence of characters. Strings are **immutable**, meaning their value cannot be changed once created in memory. Modifying a string creates a brand-new object.
- String name = "Java";

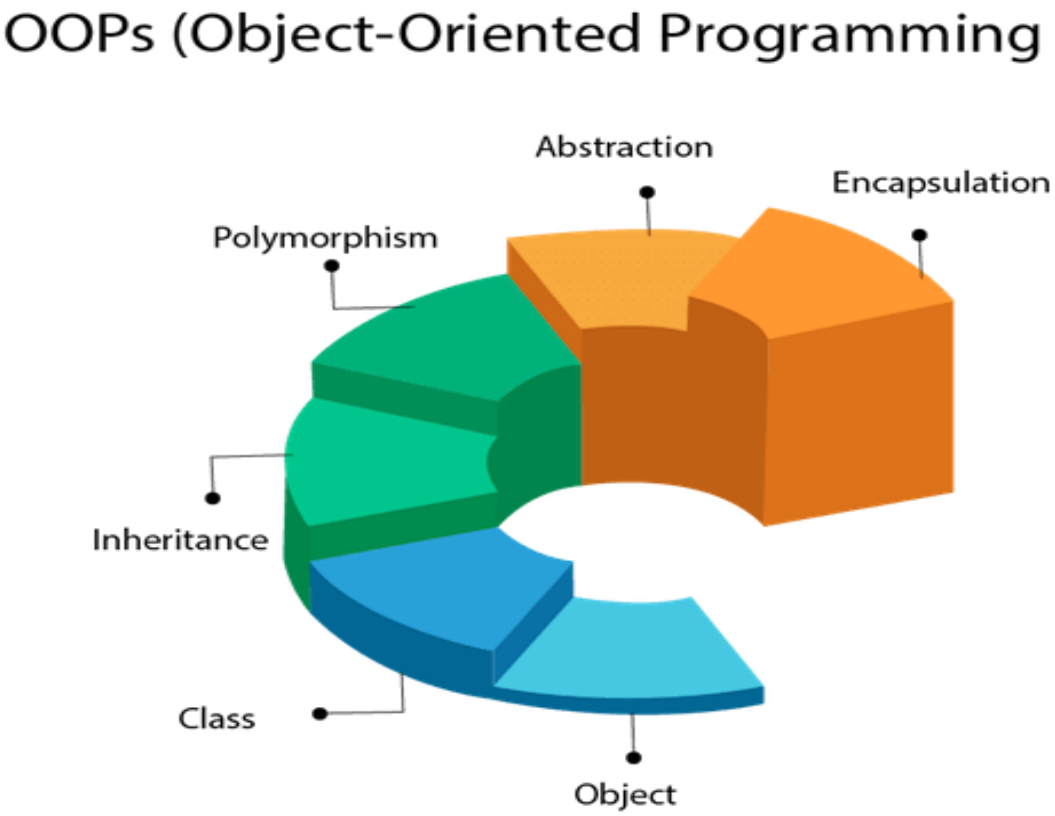
❖ String Builder and String Buffer

- Used to create **mutable** (modifiable) sequences of characters without generating redundant objects in memory:
- **StringBuilder:** Faster, but not thread-safe (preferred for single-threaded tasks).
- **StringBuffer:** Thread-safe and synchronized, but slightly slower due to overhead.

Object-Oriented Programming (OOP)

❖ Introduction to OOP

- OOP is a paradigm that organizes software design around **data or objects**, rather than functions and logic. It relies on four main pillars: Encapsulation, Inheritance, Polymorphism, and Abstraction.



❖ Classes and Objects

- **Class:** A user-defined blueprint or template used to construct objects.
- **Object:** A real-world instance of a class that possesses a state (variables) and behavior (methods).

❖ Constructors

- A special block of code automatically invoked when an object is created. It has the **same name as the class** and no explicit return type.

```
class Car {

String model;

Car(String m) { this.model = m; } // Constructor
}
```

❖ Encapsulation

- The practice of wrapping variables (data) and methods (code) together into a single unit (class). Variables are kept private and accessed securely via public getter and setter methods.

❖ Inheritance

- A mechanism where one class (child/subclass) acquires the properties and behaviors of another class (parent/superclass) using the extends keyword. This promotes code reusability.

❖ Polymorphism

- The ability of an object or method to take on multiple forms:
- **Compile-time (Static):** Achieved via **Method Overloading**.
- **Runtime (Dynamic):** Achieved via **Method Overriding** (redefining a parent class method inside a child class).

❖ Abstraction

- The process of hiding complex internal implementation details and displaying only essential functional features to the user. Achieved using abstract classes and methods.

❖ Interfaces

- A fully abstract blueprint used to achieve 100% abstraction and support **multiple inheritance** in Java. It contains only abstract methods (before Java 8) and constants. A class connects to it using the implements keyword.

❖ Packages and Access Modifiers

- **Packages:** Containers used to group related classes and interfaces together, preventing naming conflicts.
- **Access Modifiers:** Restrict scope and visibility:
  - private: Visible only within the same class.
  - default: Visible only within the same package.
  - protected: Visible within the package and by subclasses.
  - public: Accessible from anywhere in the application.
  -

Advanced Core Java

❖ Exception Handling

- A mechanism to handle runtime errors (like ArithmeticException, NullPointerException) to maintain normal application flow. It relies on five keywords: try, catch, finally, throw, and throws.
-

❖ File Handling

- Allows Java programs to read data from or write data to physical files using classes from the java.io package (like File, FileWriter, and FileReader).

❖ Collections Framework

- An architecture providing a set of ready-made classes and interfaces to manipulate groups of objects dynamically:
- **List:** Ordered collection allowing duplicates (ArrayList, LinkedList).
- **Set:** Unordered collection that bans duplicates (HashSet, TreeSet).
- **Map:** Stores data as key-value pairs (HashMap, TreeMap).

❖ Generics

- Allows classes, interfaces, and methods to accept different data types as parameters. This enforces **type safety** at compile time, eliminating the need for manual type casting.

❖ Multithreading

- The process of executing two or more threads concurrently to maximize CPU performance. Threads can be created by extending the Thread class or implementing the Runnable interface.

❖ Lambda Expressions

- Introduced in Java 8, these provide a clear, concise way to represent functional interfaces (interfaces with a single abstract method) using an arrow syntax (arguments) -> { body }.

❖ Stream API

- Introduced in Java 8, the java.util.stream package allows developers to process pipelines of data objects using a functional approach (e.g., performing filter, map, and reduce operations efficiently).

Database Connectivity

❖ Introduction to JDBC

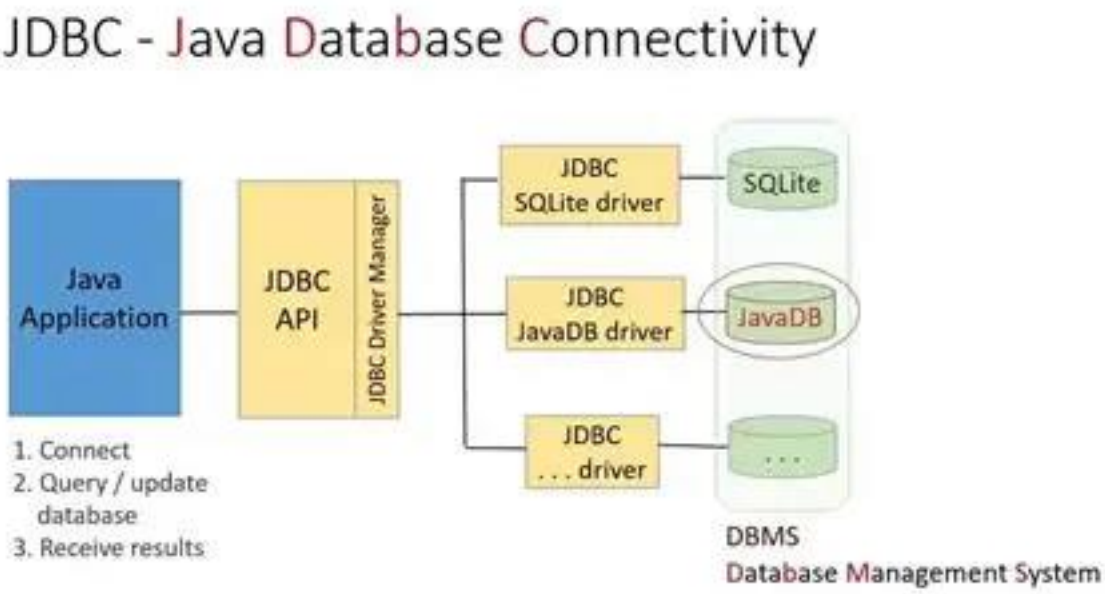
- **JDBC (Java Database Connectivity)** is a standard Java API that enables Java applications to connect to database management systems (like MySQL, Oracle, or PostgreSQL) and execute SQL queries.

❖ JDBC Architecture

- The architecture consists of two primary layers:
- **JDBC API:** Connects the Java application directly to the core JDBC Manager.
- **JDBC Driver API:** Connects the JDBC Manager to specific database drivers (e.g., MySQL Connector) to handle database-specific communication protocols.

❖ Database Connection

- Establishing a database connection requires a few foundational steps:
- Import the java.sql package.
- Load the specific database driver class (Class.forName("com.mysql.cj.jdbc.Driver")).
- Connect using the DriverManager.getConnection(url, username, password) method.



❖ CRUD Operations

- The foundational data manipulation tasks executed via SQL statements through JDBC:
- **C**reate: INSERT new rows into a database table.
- **R**ead: SELECT data from tables using a ResultSet.
- **U**pdate: UPDATE existing data records.
- **D**elete: DELETE records from tables.

❖ Java Database Application Blueprint

- Below is the standard programmatic structure for a working JDBC database query application:

```
import java.sql.Connection;

import java.sql.DriverManager;

import java.sql.ResultSet;
import java.sql.Statement;


public class JdbcExample {
```

```
public static void main(String[] args) {
String url = "jdbc:mysql://localhost:3306/my_database";
String user = "root";
String password = "password";
// Using try-with-resources to automatically close connections
try (Connection conn = DriverManager.getConnection(url, user, password);
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery("SELECT id, name FROM users")) {
while (rs.next()) {
        System.out.println("ID: " + rs.getInt("id") + ", Name: " + rs.getString("name"));
}
} catch (Exception e) {
e.printStackTrace();
}
}
}
```

- If you want to prepare for this syllabus effectively, let me know:
- Do you want **coding examples** for specific topics (like **Collections**, **Multithreading**, or **OOP pillars**)?
- Would you like me to generate a set of **practice interview questions** based on these chapters?
- Tell me how you would like to proceed!